DAV 6150 Project 1: Regression Analysis on NY State High School Dropout Metrics

Team/Author(s): <Saugat Sijapati, Sreshta Kusuru, Emmanuel Olimi Kasigazi, Bhavani Danthuri, Nicolette Mtisi>

1. Introduction

The objective of this project is to apply the data science modeling lifecycle to predict the number of student dropouts (`dropout_cnt`) within specific NY State school districts and student subgroup combinations for the 2018-2019 school year.

The target variable, `dropout_cnt`, is a non-negative integer, making this a count data regression problem. Therefore, the primary modeling techniques will be **Poisson** and **Negative Binomial** regression. These models are specifically designed for count data and will be compared against a baseline **Multiple Linear Regression (MLR)** model to evaluate their effectiveness. The project will encompass a full workflow, including data cleaning, exploratory data analysis (EDA), feature engineering, model construction, and performance evaluation.

```
# ==============================================================================
# 1. SETUP: IMPORT LIBRARIES AND LOAD DATA
# ==============================================================================

# --- Import necessary libraries ---

# Core data manipulation and numerical operations
import pandas as pd
import numpy as np

# Data visualization
import matplotlib.pyplot as plt
import seaborn as sns

# Scikit-learn for data splitting and preprocessing
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import OneHotEncoder
from sklearn.compose import ColumnTransformer

# Statsmodels for regression modeling
import statsmodels.api as sm
from statsmodels.genmod.generalized_linear_model import GLM
from statsmodels.genmod import families

# Scikit-learn for model evaluation metrics
from sklearn.metrics import mean_squared_error, r2_score

# Utility to suppress warnings for cleaner output
import warnings
warnings.filterwarnings('ignore')

# --- Define file paths and key variables ---

# The URL path to the raw CSV data file
DATA_URL = "https://github.com/nic-stack/DAV6150-/raw/refs/heads/main/Project1_Data.csv"

# The name of the target variable we want to predict
TARGET_VAR = 'dropout_cnt'

# The name of the exposure variable (cohort size), used as an offset in models
EXPOSURE_VAR = 'enroll_cnt'

# --- Load the dataset ---

# Read the data from the URL into a pandas DataFrame
df = pd.read_csv(DATA_URL)

# --- Initial Inspection ---

# Print the dimensions of the dataframe (rows, columns)
print("Shape of the initial dataset:")
print(df.shape)
print("\nFirst 5 rows of the dataset:")
# Display the first 5 rows to verify it loaded correctly
display(df.head())
```

```
# Display a concise summary of the DataFrame, including data types and non-null counts
print("\nDataset Information:")
df.info()
```

```
Shape of the initial dataset:
(73152, 29)

First 5 rows of the dataset:
```

	report_school_year	aggregation_index	aggregation_type	aggregation_name	nrc_code	nrc_desc	county_code	county_na
0	2018-19	3	District	ALBANY CITY SCHOOL DISTRICT	3	Urban-Suburban High Needs	1	ALBA
1	2018-19	3	District	ALBANY CITY SCHOOL DISTRICT	3	Urban-Suburban High Needs	1	ALBA
2	2018-19	3	District	ALBANY CITY SCHOOL DISTRICT	3	Urban-Suburban High Needs	1	ALBA
3	2018-19	3	District	ALBANY CITY SCHOOL DISTRICT	3	Urban-Suburban High Needs	1	ALBA
4	2018-19	3	District	ALBANY CITY SCHOOL DISTRICT	3	Urban-Suburban High Needs	1	ALBA

5 rows × 29 columns

```
Dataset Information:
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 73152 entries, 0 to 73151
Data columns (total 29 columns):
 #   Column                       Non-Null Count  Dtype
---  ------                       --------------  -----
 0   report_school_year           73152 non-null  object
 1   aggregation_index            73152 non-null  int64
 2   aggregation_type             73152 non-null  object
 3   aggregation_name             73152 non-null  object
 4   nrc_code                     73152 non-null  int64
 5   nrc_desc                     73152 non-null  object
 6   county_code                  73152 non-null  int64
 7   county_name                  73152 non-null  object
 8   nyc_ind                      73152 non-null  int64
 9   membership_desc              73152 non-null  object
 10  subgroup_code                73152 non-null  int64
 11  subgroup_name                73152 non-null  object
 12  enroll_cnt                   73152 non-null  object
 13  grad_cnt                     73152 non-null  object
 14  grad_pct                     73152 non-null  object
 15  local_cnt                    73152 non-null  object
 16  local_pct                    73152 non-null  object
 17  reg_cnt                      73152 non-null  object
 18  reg_pct                      73152 non-null  object
 19  reg_adv_cnt                  73152 non-null  object
 20  reg_adv_pct                  73152 non-null  object
 21  non_diploma_credential_cnt   73152 non-null  object
 22  non_diploma_credential_pct   73152 non-null  object
 23  still_enr_cnt                73152 non-null  object
 24  still_enr_pct                73152 non-null  object
 25  ged_cnt                      73152 non-null  object
 26  ged_pct                      73152 non-null  object
 27  dropout_cnt                  73152 non-null  object
 28  dropout_pct                  73152 non-null  object
dtypes: int64(5), object(24)
memory usage: 16.2+ MB
```

```
df.shape
```

```
(73152, 29)
```

2. Exploratory Data Analysis (EDA) and Data Preparation

In this section, we perform a comprehensive exploratory data analysis to understand the structure, integrity, and underlying patterns within the dataset. The primary goals are to clean the data, diagnose key statistical properties relevant to regression modeling, and prepare the dataset for the feature engineering phase.

2.1 Data Cleaning and Integrity: Handling Suppressed Data

The raw NYSED dataset uses a dash (`-`) as a placeholder for suppressed data. According to NYSED policy, data is suppressed for confidentiality when cohort sizes are too small to yield statistically reliable results.

Justification: Our models must be trained on valid, numeric data. Therefore, the first step is to convert these non-numeric placeholders into a standard `NaN` (Not a Number) format. Subsequently, any rows where our target variable, `dropout_cnt`, is `NaN` must be removed, as they provide no value for training a predictive model. We will also drop metadata columns that are redundant (e.g., code columns where text descriptions exist) or not useful as predictors (e.g., report year, which is constant).

Why are there dashes instead of numbers in some places in the New York State Report Cards

Furthermore, we will exclude the `aggregation_name` column. This feature, which identifies the specific school district, has over 680 unique categories. Including such a high-cardinality feature would dramatically increase the dimensionality of our dataset after one-hot encoding, making the model unnecessarily complex and likely leading to overfitting. While more advanced techniques like target encoding exist, for this analysis we will focus on more generalizable features like resource capacity (`nrc_desc`) and location (`nyc_ind`).

```
# ==============================================================================
# 2.1 DATA CLEANING: CONVERT SUPPRESSED VALUES AND DROP IRRELEVANT COLUMNS
# ==============================================================================

# --- 1. Convert placeholders ('-') to NaN and cast numeric columns ---

# Create a list of all columns that should be numeric (contain 'cnt' or 'pct').
numeric_cols = [col for col in df.columns if 'cnt' in col or 'pct' in col]

# Loop through each identified numeric column.
for col in numeric_cols:
    # Replace the non-numeric dash '-' with NumPy's NaN.
    # pd.to_numeric converts the column to a numeric type (e.g., float).
    # 'errors='coerce'' will turn any values that cannot be converted into NaN.
    df[col] = pd.to_numeric(df[col].replace('-', np.nan), errors='coerce')

# --- 2. Filter rows with suppressed target data ---

# Store the original number of rows to calculate how many are dropped.
original_row_count = df.shape[0]

# Drop all rows where the TARGET_VAR ('dropout_cnt') is NaN.
# .copy() is used to avoid a SettingWithCopyWarning.
df_clean = df.dropna(subset=[TARGET_VAR]).copy()

# Store the new number of rows.
cleaned_row_count = df_clean.shape[0]

# --- 3. Drop non-predictive or redundant metadata columns ---

# Define the list of columns to remove.
# We now add 'aggregation_name' due to its high cardinality.
cols_to_drop = [
    'report_school_year', 'aggregation_index', 'aggregation_type',
    'aggregation_name', #
    'nrc_code', 'county_code', 'subgroup_code', 'membership_desc'
]

# Drop the specified columns from the cleaned DataFrame.
df_clean.drop(columns=cols_to_drop, inplace=True, errors='ignore')

# --- 4. Report the results of the cleaning process ---

print(f"Shape after filtering suppressed data: {df_clean.shape}")
print(f"Total rows dropped due to suppression: {original_row_count - cleaned_row_count}")
```

```
Shape after filtering suppressed data: (39674, 21)
Total rows dropped due to suppression: 33478
```

2.2 Univariate Analysis: Target Distribution and Overdispersion

Before focusing on the target variable and feature interactions, it's a valuable EDA practice to visually inspect the distribution of each individual numeric variable. This helps us understand the scale, skewness, and presence of outliers for each potential predictor.

For each numeric column, we will generate two plots side-by-side:

- **Boxplot:** To quickly identify the median, quartiles, and outliers.
- **Histogram:** To visualize the shape of the data's distribution.

A core assumption of **Poisson Regression** is that the mean and variance of the target variable are equal (a property known as equidispersion). If the variance is significantly larger than the mean, the data is **overdispersed**.

Justification: Overdispersion violates the assumptions of the Poisson model, often leading to underestimated standard errors and an incorrect assessment of the model's fit. **Negative Binomial Regression** is a more flexible alternative that is specifically designed to handle overdispersed count data. By examining the distribution and calculating the Overdispersion Ratio (Variance / Mean), we can determine which modeling approach will be more appropriate.

```
# ============================================================================
# 2.2 COMPREHENSIVE VISUAL EDA OF NUMERIC VARIABLES (Robust Version)
# ============================================================================
# This section provides a detailed visual inspection of every numeric variable
# in the cleaned dataset to understand their distributions, skewness, and outliers.

# --- 1. Define a reusable plotting function ---
# This helper function prevents code repetition and makes our analysis scalable.
def plot_numeric_distributions(df, column_name):
    """
    Generates a side-by-side boxplot and histogram for a given numeric column.

    Args:
        df (pd.DataFrame): The DataFrame containing the data.
        column_name (str): The name of the column to visualize.
    """
    # Set up a figure with two subplots, arranged horizontally
    fig, axes = plt.subplots(1, 2, figsize=(18, 5)) # Increased width for better viewing

    # --- Boxplot (on the left subplot) ---
    sns.boxplot(x=df[column_name], ax=axes[0])
    axes[0].set_title(f'Boxplot of {column_name}')
    axes[0].set_xlabel('')
    axes[0].grid(True, linestyle='--', alpha=0.7)

    # --- Histogram (on the right subplot) ---
    sns.histplot(df[column_name], bins=40, kde=False, ax=axes[1])
    axes[1].set_title(f'Histogram of {column_name}')
    axes[1].set_xlabel('')
    axes[1].set_ylabel('Frequency')
    axes[1].grid(True, linestyle='--', alpha=0.7)

    # Display the plots for the current column
    plt.tight_layout()
    plt.show()

# --- 2. Create a list of all numeric columns to be analyzed ---
all_numeric_features = df_clean.select_dtypes(include=np.number).columns.tolist()

print("--- Generating Univariate Plots for All Numeric Features ---")
print("This systematic review is crucial for identifying skewness and outliers as noted by the team.")

# --- 3. Loop through each numeric column and generate plots + statistics ---
# This iterative process ensures every variable gets a thorough inspection.
for feature in all_numeric_features:
    print("-" * 50)
    print(f"\nAnalyzing distribution for: '{feature}'")

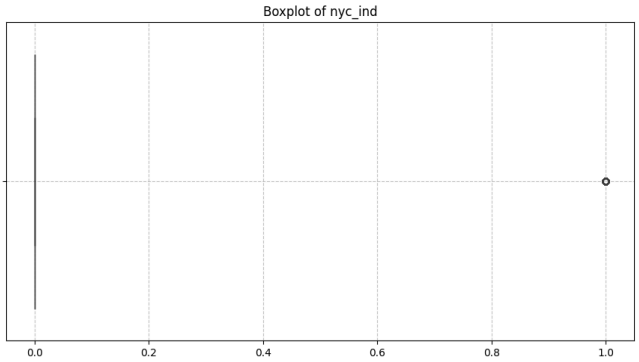
    # Print the descriptive statistics for a quantitative summary.
    print(df_clean[feature].describe())

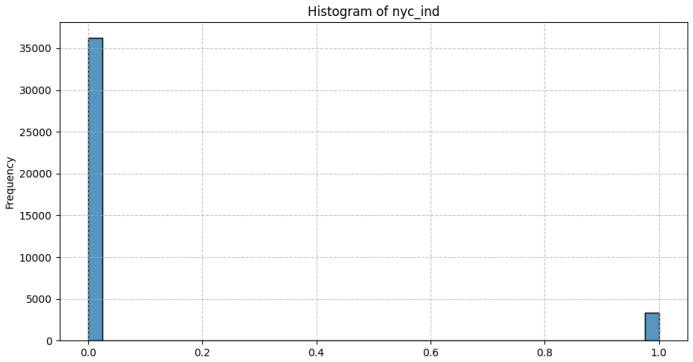
    # ========================================================================
    # ROBUSTNESS FIX: Check if the column has any valid data before plotting.
    # ========================================================================
    if df_clean[feature].count() > 0:
        # If the count of non-NaN values is greater than 0, create the plots.
```

```
            plot_numeric_distributions(df_clean, feature)
        else:
            # If the column is empty, print a message and skip plotting to avoid errors.
            print(f"\n--> SKIPPING PLOT for '{feature}' because it contains no valid data after cleaning.")
```

```
--- Generating Univariate Plots for All Numeric Features ---
This systematic review is crucial for identifying skewness and outliers as noted by the team.
--------------------------------------------------

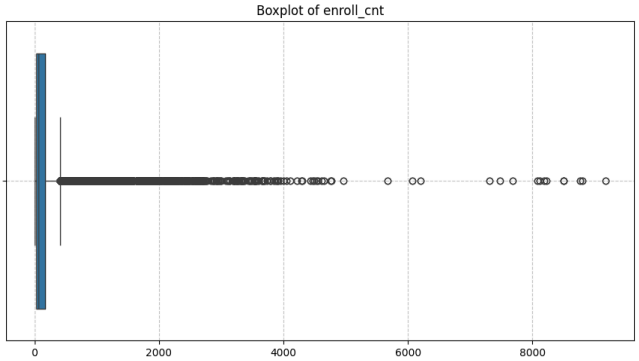
Analyzing distribution for: 'nyc_ind'
count    39674.000000
mean         0.085245
std          0.279249
min          0.000000
25%          0.000000
50%          0.000000
75%          0.000000
max          1.000000
Name: nyc_ind, dtype: float64
```

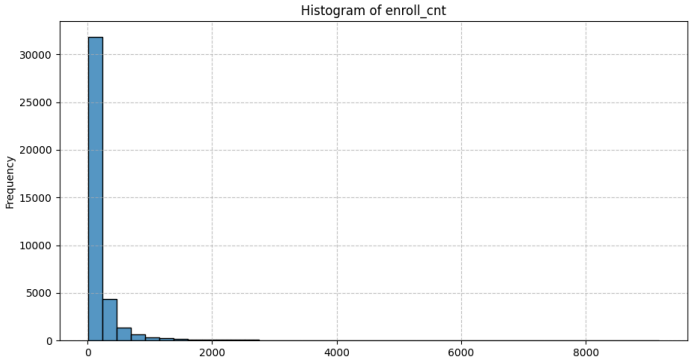





```
--------------------------------------------------

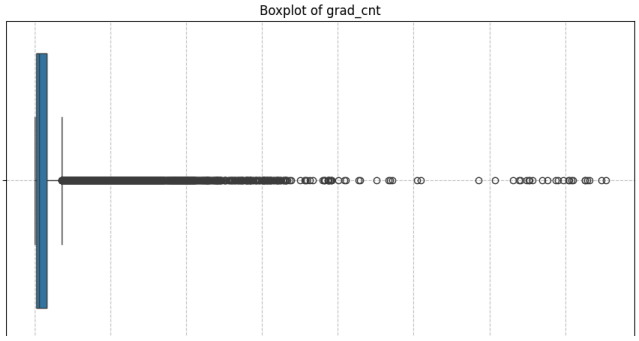
Analyzing distribution for: 'enroll_cnt'
count    39674.000000
mean       192.120079
std        439.972474
min          5.000000
25%         25.000000
50%         66.000000
75%        179.000000
max       9176.000000
Name: enroll_cnt, dtype: float64
```

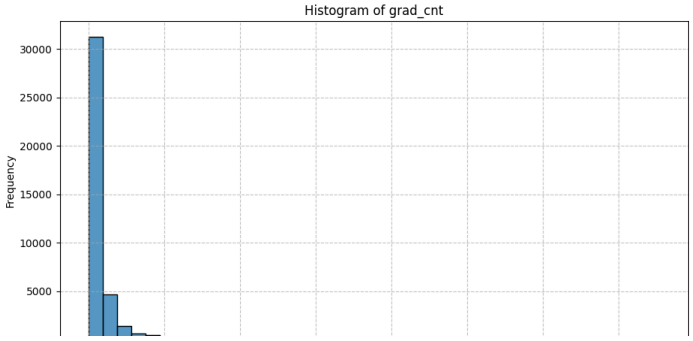


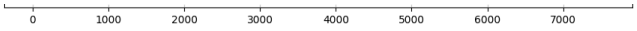
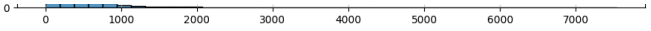



```
--------------------------------------------------

Analyzing distribution for: 'grad_cnt'
count    39674.000000
mean       161.178354
std        361.294773
min          0.000000
25%         20.000000
50%         57.000000
75%        156.000000
max       7540.000000
Name: grad_cnt, dtype: float64
```





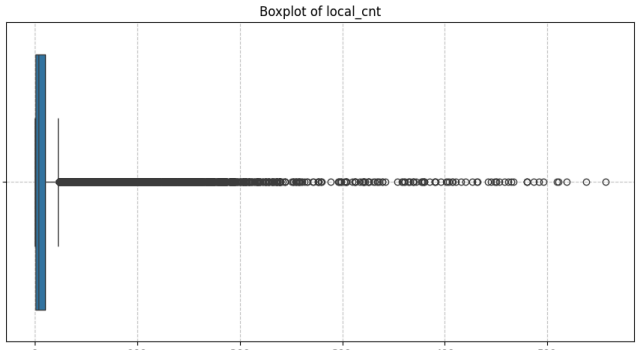



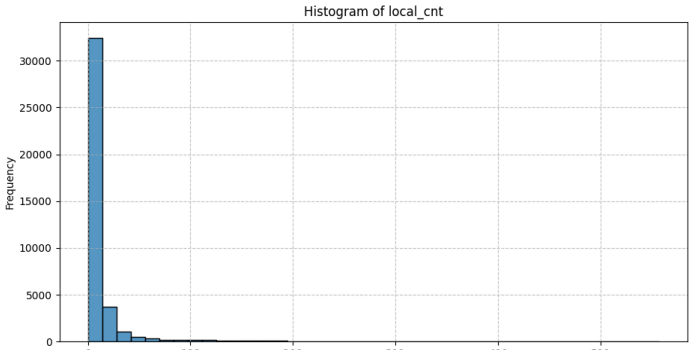
```
--------------------------------------------------

Analyzing distribution for: 'grad_pct'
count    0.0
mean     NaN
std      NaN
min      NaN
25%      NaN
50%      NaN
75%      NaN
max      NaN
Name: grad_pct, dtype: float64

--> SKIPPING PLOT for 'grad_pct' because it contains no valid data after cleaning.
--------------------------------------------------

Analyzing distribution for: 'local_cnt'
count    39674.000000
mean        12.340903
std         32.046302
min          0.000000
25%          1.000000
50%          4.000000
75%         10.000000
max        557.000000
Name: local_cnt, dtype: float64
```



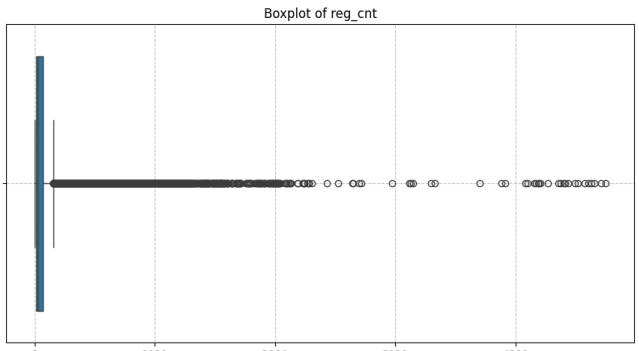


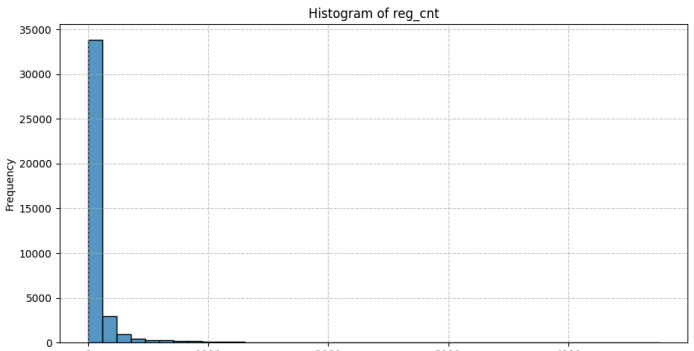

```
--------------------------------------------------

Analyzing distribution for: 'local_pct'
count    0.0
mean     NaN
std      NaN
min      NaN
25%      NaN
50%      NaN
75%      NaN
max      NaN
Name: local_pct, dtype: float64

--> SKIPPING PLOT for 'local_pct' because it contains no valid data after cleaning.
--------------------------------------------------

Analyzing distribution for: 'reg_cnt'
count    39674.000000
mean        86.804708
std        225.795826
min          0.000000
25%         10.000000
50%         27.000000
75%         69.000000
max       4752.000000
Name: reg_cnt, dtype: float64
```



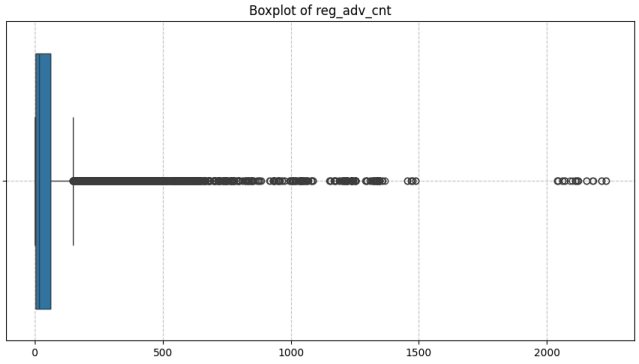


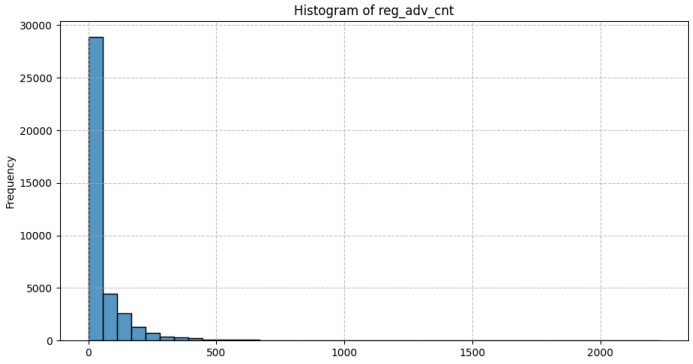
```
--------------------------------------------------

Analyzing distribution for: 'reg_pct'
count    0.0
mean     NaN
std      NaN
min      NaN
25%      NaN
50%      NaN
75%      NaN
max      NaN
Name: reg_pct, dtype: float64

--> SKIPPING PLOT for 'reg_pct' because it contains no valid data after cleaning.
--------------------------------------------------

Analyzing distribution for: 'reg_adv_cnt'
count    39674.000000
mean        62.032742
std        132.777866
min          0.000000
25%          4.000000
50%         18.000000
75%         62.000000
max       2231.000000
Name: reg_adv_cnt, dtype: float64
```



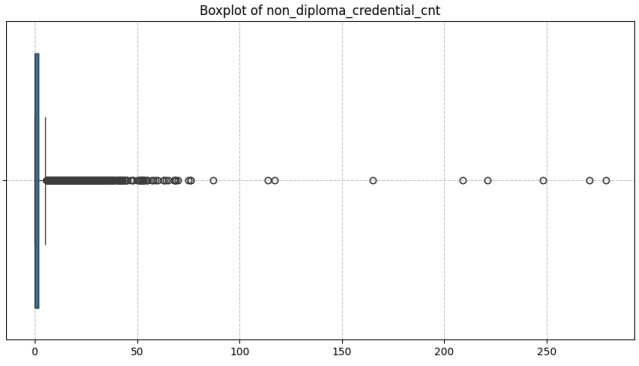


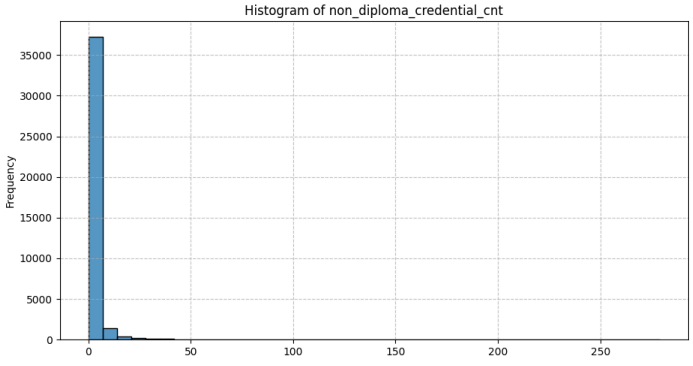

```
--------------------------------------------------

Analyzing distribution for: 'reg_adv_pct'
count    0.0
mean     NaN
std      NaN
min      NaN
25%      NaN
50%      NaN
75%      NaN
max      NaN
Name: reg_adv_pct, dtype: float64

--> SKIPPING PLOT for 'reg_adv_pct' because it contains no valid data after cleaning.
--------------------------------------------------

Analyzing distribution for: 'non_diploma_credential_cnt'
count    39674.000000
mean         1.924485
std          6.498913
min          0.000000
25%          0.000000
50%          0.000000
75%          2.000000
max        279.000000
Name: non_diploma_credential_cnt, dtype: float64
```



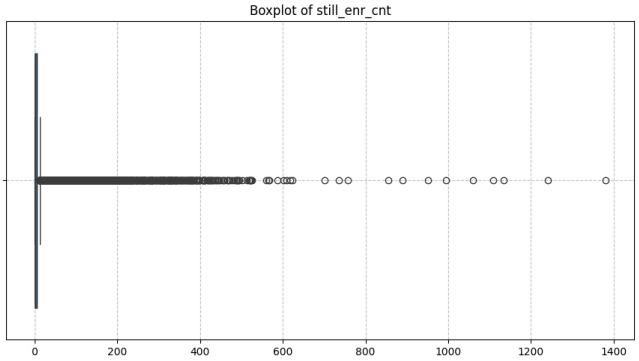


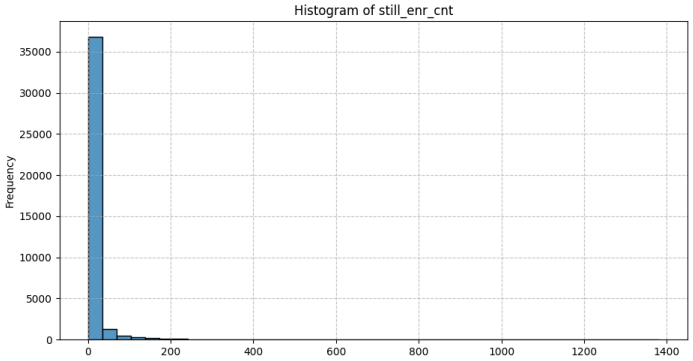

```
--------------------------------------------------
```

```
Analyzing distribution for: 'non_diploma_credential_pct'
count    0.0
mean     NaN
std      NaN
min      NaN
25%      NaN
50%      NaN
75%      NaN
max      NaN
Name: non_diploma_credential_pct, dtype: float64

--> SKIPPING PLOT for 'non_diploma_credential_pct' because it contains no valid data after cleaning.
--------------------------------------------------

Analyzing distribution for: 'still_enr_cnt'
count    39674.000000
mean        11.299516
std         40.766672
min          0.000000
25%          0.000000
50%          2.000000
75%          5.000000
max       1381.000000
Name: still_enr_cnt, dtype: float64
```



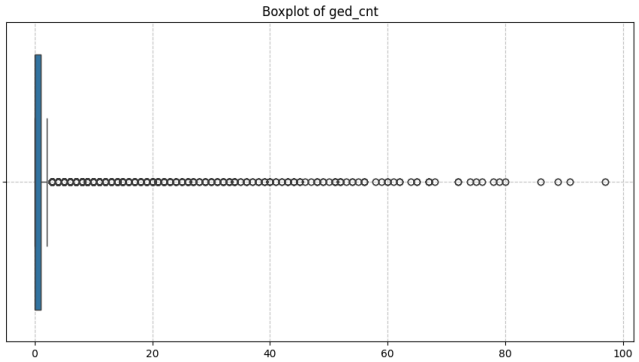


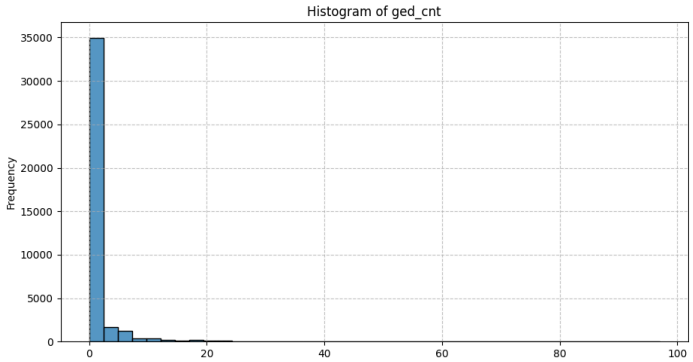

```
--------------------------------------------------

Analyzing distribution for: 'still_enr_pct'
count    0.0
mean     NaN
std      NaN
min      NaN
25%      NaN
50%      NaN
75%      NaN
max      NaN
Name: still_enr_pct, dtype: float64

--> SKIPPING PLOT for 'still_enr_pct' because it contains no valid data after cleaning.
--------------------------------------------------

Analyzing distribution for: 'ged_cnt'
count    39674.000000
mean         1.377577
std          4.949389
min          0.000000
25%          0.000000
50%          0.000000
75%          1.000000
max         97.000000
Name: ged_cnt, dtype: float64
```



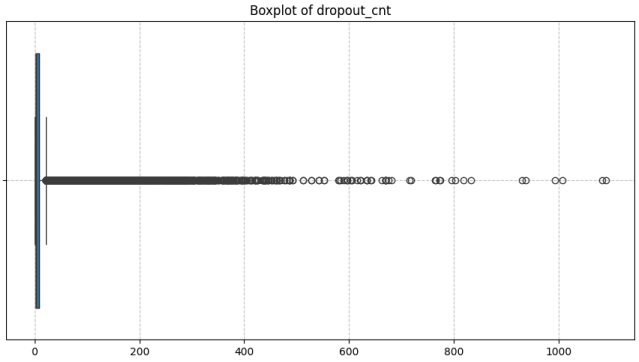


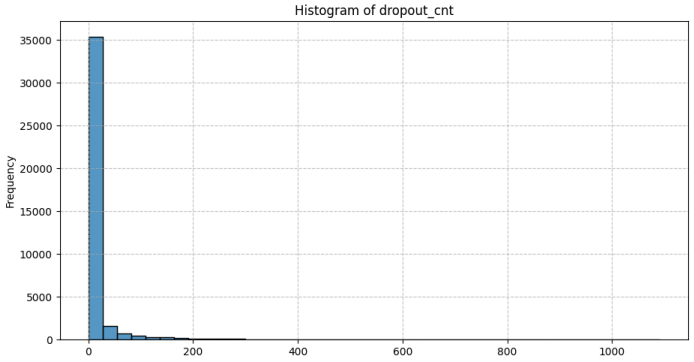

```
--------------------------------------------------
```

```
Analyzing distribution for: 'ged_pct'
count    0.0
mean     NaN
std      NaN
min      NaN
25%      NaN
50%      NaN
75%      NaN
max      NaN
Name: ged_pct, dtype: float64

--> SKIPPING PLOT for 'ged_pct' because it contains no valid data after cleaning.
--------------------------------------------------

Analyzing distribution for: 'dropout_cnt'
count    39674.000000
mean        16.239225
std         50.129834
min          0.000000
25%          1.000000
50%          3.000000
75%          9.000000
max       1091.000000
Name: dropout_cnt, dtype: float64
```






```
--------------------------------------------------

Analyzing distribution for: 'dropout_pct'
count    0.0
mean     NaN
std      NaN
min      NaN
25%      NaN
50%      NaN
75%      NaN
max      NaN
Name: dropout_pct, dtype: float64

--> SKIPPING PLOT for 'dropout_pct' because it contains no valid data after cleaning.
```